VIETNAM NATIONAL UNIVERSITY, HANOI
UNIVERSITY OF ENGINEERING AND TECHNOLOGY




# Technology Project Report

**An optimized SAT approach solving N-Queens problem**

| | |
|---|---|
| Supervisor | Dr. To Van Khanh |
| Course | INT3131 70 |
| Student | Tran Ngoc Bach - 21020170 |

**HANOI, 2024**

# Contents

**List of Tables** . . . . . **3**

**List of Figures** . . . . . **4**

**List of Algorithms** . . . . . **5**

**List of Abbreviations** . . . . . **6**

**Preface** . . . . . **7**

**1 Introduction** . . . . . **9**

1.1. Problem Definition . . . . . 9

1.2. Rules of N-Queens . . . . . 10

1.3. Applications . . . . . 10

1.4. Related works . . . . . 10

1.4.1. Traditional Approaches . . . . . 10

1.4.2. SAT encoding approach for N-Queens . . . . . 14

**2 Background** . . . . . **16**

2.1. Propositional logic . . . . . 16

2.2. Conjunctive normal form . . . . . 18

2.3. SAT and SAT solver . . . . . 18

2.4. SAT-based Encoding . . . . . 19

2.5. Optimal CNF encoding methods for AMO constraints . . . . . 20

2.5.1. Binary Encoding . . . . . 21

2.5.2. Sequential counter Encoding . . . . . 22

2.5.3. Commander Encoding . . . . . 23

2.5.4. Product Encoding . . . . . 24

**3 Methodolody** . . . . . **26**

3.1. Overall about proposed methods . . . . . 26

3.2. Optimal CNF encoding methods about N-Queens . . . . . 26
3.2.1. Binary Encoding . . . . . 27
3.2.2. Commander Encoding . . . . . 29
3.2.3. Sequential counter Encoding . . . . . 30
3.2.4. Product Encoding . . . . . 32

**4 Experiments & Result . . . . . 34**
4.1. Datasets . . . . . 34
4.2. Experimental Environments . . . . . 35

**5 Conclusion . . . . . 38**

# List of Tables

4.1 Configuration of the experimental environment . . . . . 35
4.2 The runtime of different SAT encoding methods (seconds) . . . . . 35

# List of Figures

1.1 Right solution . . . . . 9
1.2 Wrong solution . . . . . 9

2.1 SAT Solver Execution Flow . . . . . 20

4.1 Comparing the number of clauses of different CNF encoding methods . 36
4.2 Comparing the number of variables of different CNF encoding methods 37

# List of Algorithms

1 Solve N-Queens Problem . . . . . 11
2 Solve N-Queens Problem using Branch and Bound . . . . . 12

# List of Abbreviations

| Abbreviations | Full Form |
|---|---|
| ALO | At-Least-One Constraint |
| AMO | At-Most-One Constraint |
| CDCL | Conflict-Driven Clause Learning |
| CNF | Conjunctive Normal Form |
| DPLL | Davis-Putnam-Logemann-Loveland |
| PBLib | Pseudo-Boolean Optimization Library |
| SAT | Boolean Satisfiability Problem |
| UNSAT | Unsatisfiable |

# Preface

Formal methods [5] are mathematically based techniques used for specifying, developing, and verifying software and hardware systems. These methods help in precisely defining system behavior, reducing ambiguity and errors in system design. They are particularly useful in critical systems where correctness and reliability are paramount. Formal methods involve the use of formal languages and logical reasoning to ensure that systems meet their specifications. Although they can be complex, their application can significantly improve the quality and safety of systems.

Constraint Programming [2] is a programming paradigm where relationships between variables are expressed as constraints, which must be satisfied for the solution to be valid. It is particularly effective for solving combinatorial problems where the goal is to find values for variables that satisfy all the given constraints. SAT [14] as known as Boolean Satisfiability is a computational approach that determines whether a Boolean formula can be satisfied by some assignment of truth values to its variables. It is a foundational problem in computer science with applications in areas like verification, artificial intelligence, and optimization. SAT solvers have become highly efficient, enabling the resolution of complex real-world problems by transforming them into SAT instances.

Currently, many puzzles are being effectively solved using SAT encoding, yielding good results, fast computation times, and the ability to handle highly complex problems, including the N-Queens problem [17]. The N-Queens problem has been addressed using various methods such as backtracking, branch and bound, and SAT [9]. However, these methods face significant challenges in terms of computational complexity and solving time as the input size increases. In this report, I propose applying four optimal CNF Encoding methods (to be presented in *Chapter 2*) to increase the number of variables while reducing the number of clauses, thereby improving the time and performance in solving the N-Queens problem.

The report is organized as follows:

**Chapter 1:** Introduction about N-Queens problem and its related works.

**Chapter 2:** Background about SAT and SAT solver.

**Chapter 3:** Applying optimal SAT encoding into N-Queens problem.

**Chapter 4:** Experiments and performance evaluation among the methods.

**Chapter 5:** Conclusion and discussion about possible future directions.

# Chapter 1

# Introduction

## 1.1 Problem Definition

The n-queens problem is a classic puzzle in the field of artificial intelligence and combinatorial mathematics. It involves placing n queens on an n x n chessboard in such a way that no two queens can attack each other. This means that no two queens can share the same row, column, or diagonal. First proposed in the 19th century [13], the problem has become a significant topic of study due to its simplicity and broad applications in areas such as scheduling, optimization, and algorithm design.

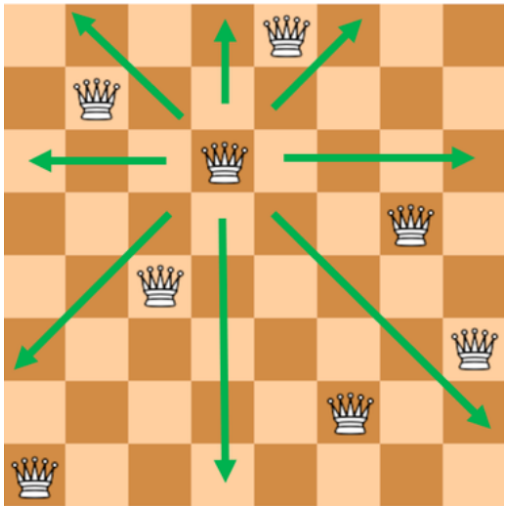


Figure 1.1: Right solution

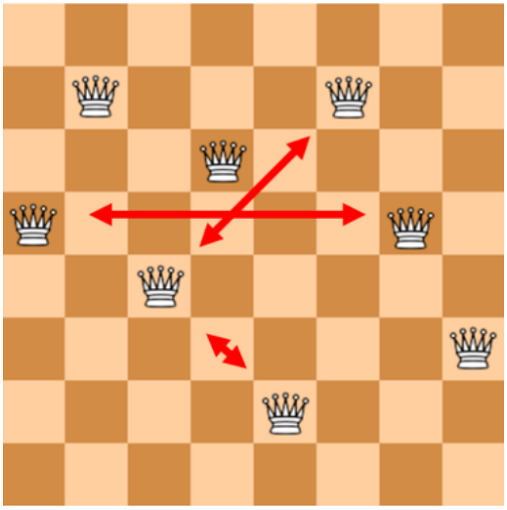


Figure 1.2: Wrong solution

The Figure 1.1 and Figure 1.2 [4] show the difference between a valid solution and an invalid solution for the n-queens problem. In the valid solution, no queens are attacking each other as is explained for queen in D3. In the invalid solution, there are 4 queens attacking each other (A4, C5, E7, F6, and G4).

## 1.2 Rules of N-Queens

The problem is set on an n x n chessboard, where n is a positive integer. There are n queens to be placed on the board.The main objective is to place all n queens on the chessboard so that no two queens threaten each other. This means:

- Rule 1: No two queens can be placed in the same row.
- Rule 2: No two queens can be placed in the same column.
- Rule 3: No two queens can be placed on the same diagonal.

The problem becomes increasingly complex as n increases, with the number of possible arrangements growing factorially. Solutions to the n-queens problem can be found using various approaches, including backtracking, heuristic algorithms, and constraint satisfaction techniques.

## 1.3 Applications

N-Queens (and the extended hyper-queen model) remarkably represents a significant model for practical task scheduling and assignment problems, computer resource management (deadlock prevention and register allocation), VLSI (Very Large Scale Integration) circuit design, air traffic control, communication system design, robot placement for maximum sensor coverage, permutation problems etc [19]. It can also be used to solve problems that involve permutations, such as the traveling salesman problem [10, 8].

## 1.4 Related works

### 1.4.1 Traditional Approaches

#### Back-tracking

*Back-tracking* approach [6] involves placing queens sequentially in various columns, beginning with the leftmost column. For each column where a queen is placed, we verify whether it conflicts with any previously placed queens. If a row is found in the current column where no conflicts occur, that row and column are marked as part of the solution. If no such row is available due to conflicts, we backtrack and indicate that the placement is not feasible.

**Algorithm 1** Solve N-Queens Problem

```
1: function solveNQueens(N)
2:     Create an array board of size N initialized to -1        ▷ Array to store the positions of queens
3:     Call solveNQueensUtil(board, 0, N)     ▷ Start solving the problem from the first row
4: end function
5: function solveNQueensUtil(board, row, N)
6:     if row = N then
7:         Print the solution using printBoard(board) ▷ All queens are placed, print the board
8:         return True
9:     end if
10:    for col = 0 to N-1 do
11:        if isSafe(board, row, col, N) then
12:            board[row] = col                    ▷ Place queen at (row, col)
13:            if solveNQueensUtil(board, row + 1, N) then
14:                return True
15:            end if
16:            board[row] = -1      ▷ Backtrack: Remove the queen if placing it didn't lead to a solution
17:        end if
18:    end for
19:    return False      ▷ Return false if no valid position for the queen was found
20: end function
```

```
21: function isSafe(board, row, col, N)
22:     for i = 0 to row-1 do
23:         if board[i] = col then
24:             return False          ▷ Check if another queen is in the same column
25:         end if
26:     end for
27:     for i = 0 to row-1 do
28:         if board[i] - i = col - row then
29:             return False                          ▷ Check the main diagonal
30:         end if
31:     end for
32:     for i = 0 to row-1 do
33:         if board[i] + i = col + row then
34:             return False                      ▷ Check the secondary diagonal
35:         end if
36:     end for
37:     return True                          ▷ Return true if no conflicts found
38: end function
```

### Branch And Bound

*Branch And Bound* approach [7] involves placing queens sequentially in each column, beginning with the leftmost one. After positioning a queen in a column, we verify if it conflicts with any previously placed queens. If a row in the current column is free from conflicts, we designate this row and column as part of the solution. If no such row is available due to conflicts, we backtrack and return false.

**Algorithm 2** Solve N-Queens Problem using Branch and Bound

```
1: Define N ← 8
2: function printSolution(board)
3:     for row = 0 to N-1 do
4:         for col = 0 to N-1 do
5:             Print board[row][col] with "  "
6:         end for
7:         Print newline
8:     end for
9: end function
10: function isSafe(row, col, slashCode, backslashCode, rowLookup, slashCode-
    Lookup, backslashCodeLookup)
11:     if slashCodeLookup[slashCode[row][col]] or backslashCode-
    Lookup[backslashCode[row][col]] or rowLookup[row] then
12:         return False
13:     else
14:         return True
15:     end if
16: end function
```

```
function solveNQueensUtil(board, col, slashCode, backslashCode, rowLookup, slashCodeLookup, backslashCodeLookup)
    for row = 0 to N-1 do
        if isSafe(row, col, slashCode, backslashCode, rowLookup, slashCodeLookup, backslashCodeLookup) then
            board[row][col] ← 1
            rowLookup[row] ← True
            slashCodeLookup[slashCode[row][col]] ← True
            backslashCodeLookup[backslashCode[row][col]] ← True
            if solveNQueensUtil(board, col + 1, slashCode, backslashCode, rowLookup, slashCodeLookup, backslashCodeLookup) then
                return True
            end if
            board[row][col] ← 0
            rowLookup[row] ← False
            slashCodeLookup[slashCode[row][col]] ← False
            backslashCodeLookup[backslashCode[row][col]] ← False
        end if
    end for
    return False
end function
function solveNQueens
    board ← an N × N matrix initialized to 0
    slashCode ← an N × N matrix initialized to 0
    backslashCode ← an N × N matrix initialized to 0
    rowLookup ← array of size N initialized to False
    x ← 2 × N − 1
    slashCodeLookup ← array of size x initialized to False
    backslashCodeLookup ← array of size x initialized to False
    for rr = 0 to N-1 do
        for cc = 0 to N-1 do
            slashCode[rr][cc] ← rr + cc
            backslashCode[rr][cc] ← rr - cc + (N - 1)
        end for
    end for
    if solveNQueensUtil(board, 0, slashCode, backslashCode, rowLookup, slashCodeLookup, backslashCodeLookup) = False then
        Print "Solution does not exist"
        return False
    end if
    printSolution(board)
    return True
end function
```

### 1.4.2 SAT encoding approach for N-Queens

In this section, an 4x4 board is taken as an example for ilustration.

| | | | |
|---|---|---|---|
| **00** | **01** | **02** | **03** |
| **10** | **11** | **12** | **13** |
| **20** | **21** | **22** | **23** |
| **30** | **31** | **32** | **33** |

Variables:

- Introduce $B_{ij}$ for $0 \leq i, j < n$.
- $B_{ij}$ is true if queen is at position $(i, j)$ and it is false otherwise. For example, if $B_{23}$ is true then there is a queen at position (2,3) on the board.

Contraints:

(a) Exactly one queen per row

We can split this constraint into two sub-constraints: "at most one" and "at least one" queen in each row.

*At most one queen in each row*

$$\bigwedge_{i=0}^{n-1} \bigwedge_{j=0}^{n-2} \bigwedge_{k=j+1}^{n-1} (\neg B_{ij} \vee \neg B_{ik}) \tag{1.1}$$

For example: $(\neg B_{00} \vee \neg B_{01}) \wedge (\neg B_{00} \vee \neg B_{02}) \wedge (\neg B_{00} \vee \neg B_{03}) \wedge ... \wedge (\neg B_{21} \vee \neg B_{22}) \wedge (\neg B_{20} \vee \neg B_{23}) \wedge ... \wedge (\neg B_{32} \vee \neg B_{33})$

*At least one queen in each row*

$$\bigwedge_{i=0}^{n-1} (B_{i0} \vee B_{i1} \vee \cdots \vee B_{in-1}) \tag{1.2}$$

For example: $B_{00} \vee B_{01} \vee B_{02} \vee B_{03} \vee ... \vee B_{20} \vee B_{21} \vee ... \vee B_{33}$

$=>$ Exactly one queen in each row: $(\neg B_{00} \vee \neg B_{01}) \wedge (\neg B_{00} \vee \neg B_{02}) \wedge (\neg B_{00} \vee \neg B_{03}) \wedge ... \wedge (\neg B_{21} \vee \neg B_{22}) \wedge (\neg B_{20} \vee \neg B_{23}) \wedge ... \wedge (\neg B_{32} \vee \neg B_{33}) \wedge (B_{00} \vee B_{01} \vee B_{02} \vee B_{03} \vee ... \vee B_{20} \vee B_{21} \vee ... \vee B_{33})$

(b) Exactly one queen per column

As same as above, this contraint can be divided into sub-constraints: "at most one" and "at least one" queen in each column.

*At most one queen in each column*

$$\bigwedge_{j=0}^{n-1} \bigwedge_{i=0}^{n-2} \bigwedge_{k=i+1}^{n-1} (\neg B_{ij} \vee \neg B_{kj}) \tag{1.3}$$

For example: $(\neg B_{00} \vee \neg B_{10}) \wedge (\neg B_{00} \vee \neg B_{20}) \wedge (\neg B_{00} \vee \neg B_{30}) \wedge ... \wedge (\neg B_{12} \vee \neg B_{22}) \wedge (\neg B_{02} \vee \neg B_{32}) \wedge ... \wedge (\neg B_{23} \vee \neg B_{33})$

*At least one queen in each column*

$$\bigwedge_{j=0}^{n-1} \left(B_{0j} \vee B_{1j} \vee \cdots \vee B_{(n-1)j}\right) \tag{1.4}$$

For example: $B_{00} \vee B_{10} \vee B_{20} \vee B_{30} \vee ... \vee B_{02} \vee B_{12} \vee ... \vee B_{33}$

$=>$ Exactly one queen in each column: $(\neg B_{00} \vee \neg B_{10}) \wedge (\neg B_{00} \vee \neg B_{20}) \wedge (\neg B_{00} \vee \neg B_{30}) \wedge ... \wedge (\neg B_{12} \vee \neg B_{22}) \wedge (\neg B_{02} \vee \neg B_{32}) \wedge ... \wedge (\neg B_{23} \vee \neg B_{33}) \wedge B_{00} \vee B_{10} \vee ... \vee B_{33}$

(c) At most one queen on diagonal

About this constraint, we will consider about the main diagonal and anti-diagonal and the lines parralel to them respectively.

*At most one queen in the main diagonal and their parallel lines*

$$\bigwedge_{k=1-n}^{n-1} \bigwedge_{\substack{0 \le i < i' \le n \\ 0 \le j < j' \le n \\ i-j=i'-j'=k}} (\neg B_{ij} \vee \neg B_{i'j'}) \tag{1.5}$$

For example: $(\neg B_{02} \vee \neg B_{13}) \wedge (\neg B_{01} \vee \neg B_{12}) \wedge ... \wedge (\neg B_{11} \vee \neg B_{22}) \wedge ... \wedge (\neg B_{20} \vee \neg B_{31})$

*At most one queen in the anti-diagonal and their parallel lines*

$$\bigwedge_{k=1}^{2n-3} \bigwedge_{\substack{0 \le i < i' \le n \\ 0 \le j < j' \le n \\ i+j=i'+j'=k}} (\neg B_{ij} \vee \neg B_{i'j'}) \tag{1.6}$$

For example: $(\neg B_{01} \vee \neg B_{10}) \wedge (\neg B_{02} \vee \neg B_{11}) \wedge ... \wedge (\neg B_{03} \vee \neg B_{21}) \wedge ... \wedge (\neg B_{23} \vee \neg B_{32})$

# Chapter 2

# Background

## 2.1 Propositional logic

Propositional logic [18], also referred to as sentential logic or statement logic, is a field within logic that explores the methods of combining and modifying entire propositions, statements, or sentences to create more complex ones. It also examines the logical relationships and properties that arise from these methods. In this type of logic, the simplest statements are treated as indivisible units, so it does not focus on the logical properties and relations of parts within statements that are not complete statements on their own, like subjects and predicates.

The most extensively studied aspect of propositional logic is classical truth-functional propositional logic. This area focuses on logical operators and connectives used to form complex statements, where the truth-value of these complex statements is determined entirely by the truth-values of the simpler statements that constitute them. It assumes that every statement is either true or false, but not both. Nonetheless, there are other forms of propositional logic that consider additional truth-values or involve connectives that form statements with truth-values influenced by factors beyond the simple truth-values of the components, such as necessity, possibility, or the relationships between the components.

The signs $\wedge$, $\vee$, $\rightarrow$, and $\neg$ correspond, respectively, to the truth-functions of *conjunction*, *disjunction*, *implication* and *negation*. We shall consider these individually.

- **Negation**
  - The negative statement of A is considered as true if A is false, and false if A is true.

– Symbol: ¬ A

– Truth table:

| $A$ | $B$ | $A \rightarrow B$ |
|---|---|---|
| $T$ | $T$ | $T$ |
| $T$ | $F$ | $F$ |
| $F$ | $T$ | $T$ |
| $F$ | $F$ | $F$ |

- **Conjunction:**

  – The conjunction between two statements A and B is true if both A and B is true, and is false if either A or B is false or both are false.

  – Symbol: $A \wedge B$ (AND)

  – Truth table:

| $A$ | $B$ | $A \wedge B$ |
|---|---|---|
| $T$ | $T$ | $T$ |
| $T$ | $F$ | $F$ |
| $F$ | $T$ | $F$ |
| $F$ | $F$ | $F$ |

- **Disjunction**

  – The disjunction between two statements A and B is true if either A or B is true, and is false if both A and B are false.

  – Symbol: $A \vee B$ (OR)

  – Truth table:

| $A$ | $B$ | $A \vee B$ |
|---|---|---|
| $T$ | $T$ | $T$ |
| $T$ | $F$ | $T$ |
| $F$ | $T$ | $T$ |
| $F$ | $F$ | $F$ |

- **Implication**

– An implication (if A, then B) is false only if A is true and B is false. The statement is incorrect only when the premise is accurate and the outcome is not.

– Symbol: A $\rightarrow$ B

– Truth table:

| $A$ | $B$ | $A \rightarrow B$ |
|---|---|---|
| $T$ | $T$ | $T$ |
| $T$ | $F$ | $F$ |
| $F$ | $T$ | $T$ |
| $F$ | $F$ | $F$ |

## 2.2 Conjunctive normal form

Conjunctive Normal Form (CNF) is a method in Boolean logic where formulas are represented as a conjunction of one or more clauses where a clause is a disjunction of literals. CNF is particularly helpful for automated theorem proving.

*Example:* $(P \vee Q) \wedge (Q \vee R) \wedge (\neg P \vee Q \vee \neg R)$ is a CNF

Every propositional formula can be transformed into a CNF formula based on propositional logic transformation principles.

## 2.3 SAT and SAT solver

Boolean Satisfiability, commonly referred to as SAT is a fundamental problem in computational theory and logic. It involves determining whether there exists an assignment of truth values (true/false) to variables in a logical formula that makes the formula true. This formula is typically represented in conjunctive normal form (CNF), where it is composed of a conjunction of clauses, with each clause being a disjunction of literals. A SAT Solver is an algorithmic tool designed to solve SAT problems. It explores possible assignments systematically or heuristically to find a satisfying assignment or prove that none exists. SAT Solvers have broad applications in fields such as artificial intelligence, verification, and optimization, due to their ability to handle complex logical conditions efficiently. There are many SAT solvers available, such as *Glucose* [1], *MiniSat* [15], *Lingeling* [3], *Sat4j* [12], and many more. These solvers have different strengths and weaknesses, making them suitable for different types of problems.

SAT has been proven to be an NP-complete problem, known for requiring algorithms with exponential worst-case complexity to solve. However, efficient and scalable algorithms for SAT have been developed in the 2000s, significantly advancing the capability to automatically solve instances involving tens of thousands of variables and millions of clauses. One of the algorithms used by SAT solvers is DPLL (Davis-Putnam-Logemann-Loveland), introduced in 1961. It is based on a backtracking algorithm that exhaustively searches all possible value assignments by assigning potential values to logical variables and then checking the results to find a solution to the problem. Additionally, modern SAT solving methods (developed in the 2000s) have introduced the CDCL (Conflict Driven Clause Learning) algorithm, which, combined with DPLL, has led to highly efficient SAT solvers applicable in many fields.

The SAT solver takes a clause in CNF as input and outputs whether the formula is satisfiable (SAT) or unsatisfiable (UNSAT). It also provides the potential values that make the formula true.

- **INPUT:** A propositional logic formula, represented as in the form of conjunctive normal form.
- **OUTPUT:**
  - SAT: if there exists an assignment of truth values (true/false) to the propositional logic variables that makes the original propositional logic formula true.
  - UNSAT: when all possible assignments of truth values (true/false) to the propositional logic variables make the original propositional logic formula false.

*For example:*

$$F_1 = (\neg x_1 \vee x_3) \wedge (x_2 \vee x_3) \wedge (\neg x_2 \vee x_1)$$
$$F_2 = \neg x_1 \wedge (x_1 \vee x_2) \wedge (x_1 \vee \neg x_2)$$

The formula $F_1$ is true when $x_1 =$ false, $x_2 =$ false, $x_3 =$ true so F is SAT.
However, $F_2$ is UNSAT because it is false, irrespective of any set of values $(x_1, x_2)$

## 2.4 SAT-based Encoding

SAT Encoding is an approach of representing problems through propositional logic formulas and applying SAT Solvers to solve these propositional logic formulas.

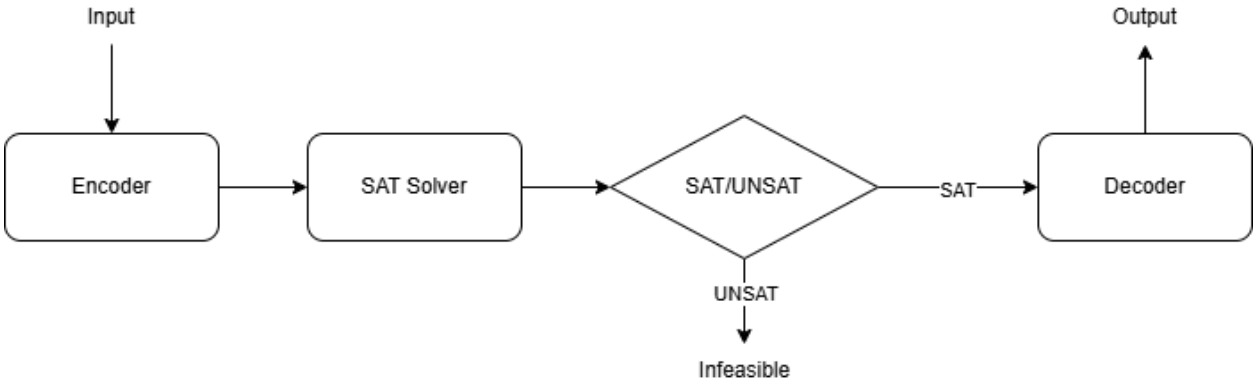



Figure 2.1: SAT Solver Execution Flow

As shown in *Figure 2.1*, it begins with problem formulation, where the problem is defined in terms of logical constraints. This problem is then encoded into a propositional logic formula and converted into Conjunctive Normal Form (CNF), a standardized format required by SAT solvers. Then the encoder use the input data to generate a data file that includes the count of variables, the count of clauses, and the CNF clauses themselves. The data file is inputted into a SAT solver, which processes it to determine if there is an assignment of truth values to variables that satisfies all clauses. The output from the SAT solver will be either SAT, indicating that a satisfying assignment was found, or UNSAT, meaning no such assignment exists. If the result is SAT, the satisfying assignment is intepreted by the decoder and return the solution for the original problem.

## 2.5 Optimal CNF encoding methods for AMO constraints

At-Most-One (AMO) constraints are a type of constraint that restricts the number of variables that can be true to at most one. AMO constraints are a common type of constraint in many optimization problems. There are several methods to encode AMO constraints into CNF formulas, including *Binomial*, *Binary*, *Sequential Encounter*, *Commander*, and *Product* encoding. We illustrated *Binomial* encoding in the section 1.4.2. This encoding create a lot of clauses, with the complexity of $\binom{n}{2}$ where $n$ is the number of variables. This is not an optimal encoding method for AMO constraints. We will introduce some optimal encoding methods for AMO constraints.

### 2.5.1 Binary Encoding

**Idea[11]:**

- Original variables: $X_i$ $(i = 1, 2..., n)$
- New $k$ variables $Y_1, Y_2, \ldots, Y_k$ $(k = \lceil \log_2 n \rceil)$
- $X_i$ corresponding a binary string (000, 011, 0110...)
- Clauses: $X_i \leftrightarrow \varphi(i,1) \wedge \varphi(i,2) \wedge \ldots \wedge \varphi(i,k)$ where
  - $\varphi(i,j)$ denotes for $Y_j$ if $j$-th digit in binary string is 1
  - $\varphi(i,j)$ denotes for $\neg Y_j$ if $j$-th digit in binary string is 0
- Requires: $O(\log_2 n)$ new variables, $O(n \log_2 n)$ clauses

***AMO Encoding:***

$$\bigwedge_{i=1}^{n} \bigwedge_{j=1}^{k=\lceil \log_2 n \rceil} (\neg X_i \vee \varphi(i,j))$$

***ALO Encoding:***

$$X_1 \vee X_2 \vee \ldots \vee X_n$$

**Example:** n = 3 $(X_1, X_2, X_3)$
$\lceil \log_2 3 \rceil = 2 => $ two more new variables $Y_1, Y_2$ required

| | |
|---|---|
| $X_1$ corresponding to "00" | $X_1 \leftrightarrow \neg Y_1 \wedge \neg Y_2$ |
| $X_2$ corresponding to "01" | $X_2 \leftrightarrow Y_1 \wedge \neg Y_2$ |
| $X_3$ corresponding to "10" | $X_3 \leftrightarrow \neg Y_1 \wedge Y_2$ |

- AMO Encoding:

$$(X_1 \rightarrow \neg Y_1 \wedge \neg Y_2) \wedge (X_2 \rightarrow Y_1 \wedge \neg Y_2) \wedge (X_3 \rightarrow \neg Y_1 \wedge Y_2)$$

- ALO Encoding:

$$X_1 \vee X_2 \vee X_3$$

### 2.5.2 Sequential counter Encoding

**Idea[11]:**

- Original variables: $X_1, X_2, \ldots, X_n$
- New $(n-1)$ variables $S_1, S_2, \ldots, S_{n-1}$
- $S_i$ = True if existing a truth variable in $(X_1, X_2, \ldots, X_i)$
- *If* $S_i$ = True *then* $X_{i+1}$ *false*
- Requires: $(n-1)$ new variables, $(3n-4)$ clauses

***AMO Encoding:*** $X_1 + X_2 + \ldots + X_n \leq 1$

$$X_1 \rightarrow S_1$$

$$\wedge(X_i \vee S_{i-1}) \rightarrow S_i \wedge (S_{i-1} \rightarrow \neg X_i) \quad (1 < i < n)$$

$$\wedge(S_{n-1} \rightarrow \neg X_n)$$

or

$$\neg X_1 \vee S_1$$

$$\wedge \bigwedge_{i=2}^{n-1} (\neg X_i \vee S_i) \wedge \bigwedge_{i=2}^{n-1} (\neg S_{i-1} \vee S_i) \wedge \bigwedge_{i=2}^{n-1} (\neg S_{i-1} \vee \neg X_i)$$

$$\wedge(\neg S_{n-1} \vee \neg X_n)$$

***ALO Encoding:***

$$X_1 \vee X_2 \vee \ldots \vee X_n$$

**Example:** n = 4 $(X_1, X_2, X_3, X_4)$

n - 1 = 3 new required variables $(S_1, S_2, S_3)$

- AMO Encoding:

$$X_1 \rightarrow S_1 \wedge (X_2 \vee S_1) \rightarrow S_2 \wedge (S_1 \rightarrow \neg X_2) \wedge (X_3 \vee S_2) \rightarrow S_3 \wedge (S_2 \rightarrow \neg X_3) \wedge (S_3 \rightarrow \neg X_4)$$

- ALO Encoding:

$$X_1 \vee X_2 \vee X_3 \vee X_4$$

### 2.5.3 Commander Encoding

**Idea[11]:**

- Partitioning set of variables $X_1, X_2, \ldots, X_n$ into some groups of some fixed size and encoding AMO over each group
  - $X_1, X_2, \ldots, X_k \in G_1$
  - $X_{k+1}, X_{k+2}, \ldots, X_{2k} \in G_2; \ldots$
  - $X_j, \ldots, X_n \in G_g$
  - $g = \lfloor \sqrt{n} \rfloor$
- Introducing a commander variable for each group: $c_1, c_2, \ldots, c_g$

***AMO Encoding:***

- At most one (AMO) $ci$ is true. Use AMO binomial encoding for clauses: $c_1, c_2, \ldots, c_g$

$$\bigwedge_{i=1}^{g-1} \bigwedge_{j=i+1}^{g} (\neg c_i \vee \neg c_j)$$

- If a commander variable $c_i$ is true then there exists variables true in $G_i$.

$$\bigwedge_{i=1}^{g} \left( c_i \rightarrow \bigvee_{x \in G_i} x \right)$$

- If a commander variable $c_i$ is true then exactly one variable true in $G_i$. Use AMO binomial encoding for this constraint.

$$\bigwedge_{p=1}^{k-1} \bigwedge_{q=i+1}^{k} (\neg x_p \vee \neg x_q)$$

(where $k$ is the number of $x$ in a group $G_i$)

- If a commander variable $c_i$ is false then all variables false in $G_i$

$$\bigwedge_{i=1}^{g} \bigwedge_{x \in G_i} (\neg c_i \rightarrow \neg x)$$

***ALO Encoding:***

$$X_1 \vee X_2 \vee \ldots \vee X_n$$

**Example:**

For $n = 7$ $(X_1, X_2, \ldots, X_7)$, $g = \lfloor \sqrt{7} \rfloor = 2$

- AMO encoding:
  - $X_1, X_2, X_3, X_4 \in G_1 \qquad G_1 \Leftrightarrow c_1$
  - $X_5, X_6, X_7 \in G_2 \qquad G_2 \Leftrightarrow c_2$
  - $(c_1 \rightarrow \neg c_2) \wedge$

    $c_1 \rightarrow (X_1 \vee X_2 \vee X_3 \vee X_4) \wedge (X_1 \rightarrow \neg X_2) \wedge (X_1 \rightarrow \neg X_3) \wedge (X_1 \rightarrow \neg X_4) \wedge (X_2 \rightarrow \neg X_3) \wedge (X_2 \rightarrow \neg X_4) \wedge (X_3 \rightarrow \neg X_4) \wedge$

    $c_2 \rightarrow (X_5 \vee X_6 \vee X_7) \wedge (X_5 \rightarrow \neg X_6) \wedge (X_5 \rightarrow \neg X_7) \wedge (X_6 \rightarrow \neg X_7) \wedge$

    $(\neg c_1 \rightarrow \neg X_1) \wedge (\neg c_1 \rightarrow \neg X_2) \wedge (\neg c_1 \rightarrow \neg X_3) \wedge (\neg c_1 \rightarrow \neg X_4) \wedge$

    $(\neg c_2 \rightarrow \neg X_5) \wedge (\neg c_2 \rightarrow \neg X_6) \wedge (\neg c_2 \rightarrow \neg X_7)$
- ALO Encoding:

$$X_1 \vee X_2 \vee X_3 \vee \ldots \vee X_7$$

### 2.5.4 Product Encoding

**Idea[11]:**

- Decompose AMO $(X_1, X_2, \ldots, X_n)$ into: AMO $(r_1, r_2, \ldots, r_p)$ and AMO $(c_1, c_2, \ldots, c_q)$
- A grid with $n$ points $(p \times q) : p = \lceil \sqrt{n} \rceil, q = \lceil n/p \rceil$
- One point for each variable $(X_1, X_2, \ldots, X_n)$
  - $X_1 \leftrightarrow (r_1, c_1), X_2 \leftrightarrow (r_1, c_2), \ldots, X_n \leftrightarrow (r_p, c_k) \qquad k = n - (p-1)q$
- Constraints at most one row and at most one column

**AMO Encoding** $(X_1, X_2, \ldots, X_n)$**:**

- AMO $(r_1, r_2, \ldots, r_p)$ and AMO $(c_1, c_2, \ldots, c_q)$
- $X_i \to r_u \wedge c_v$ where $X_i \leftrightarrow (r_u, c_v)$

**ALO Encoding** $(X_1, X_2, \ldots, X_n)$**:**

- (i): $(X_1 \vee X_2 \vee X_3 \vee \ldots \vee X_n)$
- (ii): ALO $(r_1, r_2, \ldots, r_p)$ and ALO $(c_1, c_2, \ldots, c_q)$ and $r_u \wedge c_v \to X_i$ where $X_i \leftrightarrow (r_u, c_v)$

**Example:**

- AMO encoding:

$$p = \lceil \sqrt{n} \rceil = \lceil \sqrt{8} \rceil = 3$$

$$q = \left\lceil \frac{n}{p} \right\rceil = \left\lceil \frac{8}{3} \right\rceil = 3$$

So:

| | | |
|---|---|---|
| $X_1$ | $X_2$ | $X_3$ |
| $X_4$ | $X_5$ | $X_6$ |
| $X_7$ | $X_8$ | |

  - $(r_1 \to \neg r_2) \wedge (r_1 \to \neg r_3) \wedge (r_2 \to \neg r_3) \wedge$
    $(c_1 \to \neg c_2) \wedge (c_1 \to \neg c_3) \wedge (c_2 \to \neg c_3) \wedge$
    $(X_1 \to r_1) \wedge (X_1 \to r_1) \wedge$
    $(X_2 \to r_1) \wedge (X_2 \to c_2) \wedge$
    $(X_3 \to r_1) \wedge (X_3 \to c_3) \wedge$
    $(X_4 \to r_2) \wedge (X_4 \to c_1) \wedge$
    $(X_5 \to r_2) \wedge (X_5 \to c_2) \wedge$
    $(X_6 \to r_2) \wedge (X_6 \to c_3) \wedge$
    $(X_7 \to r_3) \wedge (X_7 \to c_1) \wedge$
    $(X_8 \to r_3) \wedge (X_8 \to c_3)$

- ALO Encoding:

$$X_1 \vee X_2 \vee X_3 \vee \ldots \vee X_8$$

# Chapter 3

# Methodolody

## 3.1 Overall about proposed methods

The *Binomial* encoding method proves effective and fast in solving the N-Queens problem for smaller board sizes. However, as $n$ increases, the number of clauses grows significantly, leading to longer computation times to obtain results. Therefore, I propose applying alternative encoding methods to increase the number of variables while reducing the number of clauses generated by the constraints.

Regarding constraints: *Exactly one (EXO) = at most one (AMO) + at least one (ALO)*. I will focus on optimizing the AMO formulas of binomial encoding, specifically the formulas 1.1, 1.3, 1.5, 1.6

## 3.2 Optimal CNF encoding methods about N-Queens

Based on the rules of the N-Queens game, a correct solution is a chessboard with N queens such that they cannot attack each other horizontally, vertically, or diagonally. In other words, on the chessboard, there is exactly one queen in each row, column, and diagonal. Therefore, the general idea of the encoding methods presented below is to define the AMO (At Most One) and ALO (At Least One) encoding formulas corresponding to each row, column, and diagonal. In these encoding methods, we still take the 4x4 board as an example.

| | | | |
|---|---|---|---|
| **00** | **01** | **02** | **03** |
| **10** | **11** | **12** | **13** |
| **20** | **21** | **22** | **23** |
| **30** | **31** | **32** | **33** |

### 3.2.1 Binary Encoding

#### Encoding Variables

*Decision Variables*:

- Introduce $B_{ij}$ for $0 \leq i, j < n$.
- $B_{ij}$ is true if queen is at position $(i, j)$ and it is false otherwise. For example, if $B_{23}$ is true then there is a queen at position (2,3) on the board.

*Auxiliary Variables:*

Use $k$ new variables $Y_1, Y_2, \ldots, Y_k$ ($k = \lceil \log_2 m \rceil$, where m is the number of positions in each row, column, and diagonal) corresponding to each row, column, and diagonal on the chessboard.

#### Encoding Contraints

(a) Exactly one queen per row

We can split this constraint into two sub-constraints: "at most one" and "at least one" queen in each row and combine them by AND.

*At most one queen in each row*

$$\bigwedge_{i=0}^{n-1} \bigwedge_{j=0}^{n-1} \bigwedge_{p=i*k}^{i*k+k-1} (\neg B_{ij} \vee \varphi(j, p)) \tag{3.1}$$

For example in $row_0$:

$$(B_{00} \rightarrow \neg Y_0 \wedge \neg Y_1) \wedge (B_{01} \rightarrow Y_0 \wedge \neg Y_1) \wedge (B_{02} \rightarrow \neg Y_0 \wedge Y_1) \wedge (B_{03} \rightarrow Y_0 \wedge Y_1)$$

*At least one queen in each row*

$$\bigwedge_{i=0}^{n-1}(B_{i0} \vee B_{i1} \vee ...B_{i,n-1}) \tag{3.2}$$

For example: $B_{00} \vee B_{01} \vee B_{02} \vee B_{03} \vee ... \vee B_{20} \vee B_{21} \vee ... \vee B_{33}$

(b) Exactly one queen per column

As same as above, this contraint can be divided into sub-constraints: "at most one" and "at least one" queen in each column and combine them by AND.

*At most one queen in each column*

$$\bigwedge_{j=0}^{n-1}\bigwedge_{i=0}^{n-1}\bigwedge_{p=j*k}^{j*k+k-1}(\neg B_{ij} \vee \varphi(i,p)) \tag{3.3}$$

For example in $column_0$:

$$(B_{00} \to \neg Y_0 \wedge \neg Y_1) \wedge (B_{10} \to Y_0 \wedge \neg Y_1) \wedge (B_{20} \to \neg Y_0 \wedge Y_1) \wedge (B_{30} \to Y_0 \wedge Y_1)$$

*At least one queen in each column*

$$\bigwedge_{j=0}^{n-1}(B_{0j} \vee B_{1j} \vee \cdots \vee B_{n-1,j}) \tag{3.4}$$

For example: $B_{00} \vee B_{10} \vee B_{20} \vee B_{30} \vee ... \vee B_{02} \vee B_{12} \vee ... \vee B_{33}$

(c) At most one queen on diagonal

About this constraint, we will consider about the main diagonal and anti-diagonal and the lines parralel to them respectively.

*At most one queen in the main diagonal and their parallel lines*

$$\bigwedge_{q=1-n}^{n-1}\bigwedge_{\substack{0\le i<i'\le n\\ 0\le j<j'\le n\\ i-j=i'-j'=q}}\bigwedge_{p=j*k}^{j*k+k-1}(\neg B_{ij} \vee \varphi(i,p)) \tag{3.5}$$

For example the diagongal $B_{00}, B_{11}, B_{22}, B_{33}$:

$$(B_{00} \to \neg Y_0 \wedge \neg Y_1) \wedge (B_{11} \to Y_0 \wedge \neg Y_1) \wedge (B_{22} \to \neg Y_0 \wedge Y_1) \wedge (B_{33} \to Y_0 \wedge Y_1)$$

*At most one queen in the anti-diagonal and their parallel lines*

$$\bigwedge_{k=1}^{2n-3} \bigwedge_{\substack{0\leq i<i'\leq n \\ 0\leq j<j'\leq n \\ i+j=i'+j'=k}} \bigwedge_{p=j*k}^{j*k+k-1} (\neg B_{ij} \vee \varphi(i,p)) \tag{3.6}$$

For example $B_{02}, B_{20}, B_{11}$:

$$(B_{02} \rightarrow \neg Y_0 \wedge \neg Y_1) \wedge (B_{20} \rightarrow Y_0 \wedge \neg Y_1) \wedge (B_{11} \rightarrow \neg Y_0 \wedge Y_1)$$

### 3.2.2 Commander Encoding

#### Encoding Variables

*Decision Variables*:

- Introduce $B_{ij}$ for $0 \leq i, j < n$.
- $B_{ij}$ is true if queen is at position $(i, j)$ and it is false otherwise. For example, if $B_{23}$ is true then there is a queen at position (2,3) on the board.

*Auxiliary Variables:*

- Introducing a commander variable for each group: $c_1, c_2, \ldots, c_g$
  - If a commander variable $c_i$ is true then exactly one variable true in $G_i$
  - If a commander variable $c_i$ is false then all variables false in $G_i$

#### Encoding Contraints

In this encoding, I will provide an example with a specific row, and the same applies to columns and diagonals.

Take $row_1$ as example:

We can split this constraint *Exactly one queen per row* into two sub-constraints: "at most one" and "at least one" queen in each row.

*At most one one queen per row*

This row has 4 positions so it is divided in to $g = \lfloor\sqrt{4}\rfloor = 2$

- $B_{00}, B_{01} \in G_1 \qquad G_1 \Leftrightarrow c_1$
- $B_{02}, B_{03} \in G_2 \qquad G_2 \Leftrightarrow c_2$

- $(c_1 \rightarrow \neg c_2) \wedge$

  $c_1 \rightarrow (B_{00} \vee B_{01}) \wedge (B_{00} \rightarrow \neg B_{01}) \wedge$

  $c_2 \rightarrow (B_{02} \vee B_{03}) \wedge (B_{02} \rightarrow \neg B_{03}) \wedge$

  $(\neg c_1 \rightarrow \neg B_{00}) \wedge (\neg c_1 \rightarrow \neg B_{01}) \wedge$

  $(\neg c_1 \rightarrow \neg B_{02}) \wedge (\neg c_1 \rightarrow \neg B_{03}) \wedge$

*At least one one queen per row*
$B_{00} \vee B_{01} \vee B_{02} \vee B_{03}$

### 3.2.3 Sequential counter Encoding

#### Encoding Variables

*Decision Variables*:

- Introduce $B_{ij}$ for $0 \leq i, j < n$.
- $B_{ij}$ is true if queen is at position $(i, j)$ and it is false otherwise. For example, if $B_{23}$ is true then there is a queen at position (2,3) on the board.

*Auxiliary Variables:*

- Introduce new variables S
- $S_i$ = True if existing a truth variable in $(X_1, X_2, \ldots, X_i)$

#### Encoding Contraints

(a) Exactly one queen per row

We can split this constraint into two sub-constraints: "at most one" and "at least one" queen in each row.

*At most one queen in each row*

$$\bigwedge_{i=0}^{n-1} \neg B_{i0} \vee S_{i0}$$
$$\wedge \bigwedge_{i=0}^{n-1} \bigwedge_{j=1}^{n-1} (\neg B_{ij} \vee S_{ij}) \wedge \bigwedge_{i=0}^{n-1} \bigwedge_{j=1}^{n-1} (\neg S_{i,j-1} \vee S_j) \wedge \bigwedge_{i=0}^{n-1} \bigwedge_{j=1}^{n-1} (\neg S_{i,j-1} \vee \neg B_{ij}) \tag{3.7}$$
$$\wedge \bigwedge_{i=0}^{n-1} (\neg S_{i,n-2} \vee B_{i,n-1})$$

For example: $(\neg B_{00} \vee \neg B_{01}) \wedge (\neg B_{00} \vee \neg B_{02}) \wedge (\neg B_{00} \vee \neg B_{03}) \wedge ... \wedge (\neg B_{21} \vee \neg B_{22}) \wedge (\neg B_{20} \vee \neg B_{23}) \wedge ... \wedge (\neg B_{32} \vee \neg B_{33})$

*At least one queen in each row*

$$\bigwedge_{i=0}^{n-1} (B_{i0} \vee B_{i1} \vee \cdots \vee B_{i,n-1}) \tag{3.8}$$

For example: $B_{00} \vee B_{01} \vee B_{02} \vee B_{03} \vee ... \vee B_{20} \vee B_{21} \vee ... \vee B_{33}$

(b) Exactly one queen per column

As same as above, this contraint can be divided into sub-constraints: "at most one" and "at least one" queen in each column.

*At most one queen in each column*

$$\bigwedge_{j=0}^{n-1} \neg B_{0j} \vee S_{0j}$$
$$\wedge \bigwedge_{j=0}^{n-1} \bigwedge_{i=1}^{n-1} (\neg B_{ij} \vee S_{ij}) \wedge \bigwedge_{j=0}^{n-1} \bigwedge_{i=1}^{n-1} (\neg S_{i-1,j} \vee S_i) \wedge \bigwedge_{j=0}^{n-1} \bigwedge_{i=1}^{n-1} (\neg S_{i-1,j} \vee \neg B_{ij}) \tag{3.9}$$
$$\wedge \bigwedge_{j=0}^{n-1} (\neg S_{n-2,j} \vee B_{n-1,j})$$

For example: $(\neg B_{00} \vee \neg B_{10}) \wedge (\neg B_{00} \vee \neg B_{20}) \wedge (\neg B_{00} \vee \neg B_{30}) \wedge ... \wedge (\neg B_{12} \vee \neg B_{22}) \wedge (\neg B_{02} \vee \neg B_{32}) \wedge ... \wedge (\neg B_{23} \vee \neg B_{33})$

*At least one queen in each column*

$$\bigwedge_{j=0}^{n-1}(B_{0j} \vee B_{1j} \vee \cdots \vee B_{n-1,j}) \tag{3.10}$$

For example: $B_{00} \vee B_{10} \vee B_{20} \vee B_{30} \vee ... \vee B_{02} \vee B_{12} \vee ... \vee B_{33}$

(c) At most one queen on diagonal

About this constraint, we will consider about the main diagonal and anti-diagonal and the lines parralel to them respectively.

*At most one queen in the main diagonal and their parallel lines*

- In this contraint, I will take a specific diagonal for to be more clear.
- Consider: $B_{00}, B_{11}, B_{22}, B_{33}$, n - 1 = 3 new required variables $(S_1, S_2, S_3)$

=> Encoding:

$$B_{00} \to S_1 \wedge (B_{11} \vee S_1) \to S_2 \wedge (S_1 \to \neg B_{11}) \wedge (B_{22} \vee S_2) \to S_3 \wedge (S_2 \to \neg B_{22}) \wedge (S_3 \to \neg B_{33})$$

*At most one queen in the anti-diagonal and their parallel lines*

- In this contraint, I will take a specific diagonal for to be more clear.
- Consider: $B_{02}, B_{20}, B_{11}$, n - 1 = 2 new required variables $(S_1, S_2)$

=> Encoding:

$$B_{02} \to S_1 \wedge (B_{20} \vee S_1) \to S_2 \wedge (S_1 \to \neg B_{20}) \wedge (S_2 \to B_{11})$$

### 3.2.4 Product Encoding

#### Encoding Variables

*Decision Variables*

*Decision Variables*:

- Introduce $B_{ij}$ for $0 \leq i, j < n$.
- $B_{ij}$ is true if queen is at position $(i, j)$ and it is false otherwise. For example, if $B_{23}$ is true then there is a queen at position (2,3) on the board.

*Auxiliary Variables:*

- Introduce variables r and c.

**Encoding Contraints**

*AMO Encoding* In this encoding, I will provide an example with a specific row, and the same applies to columns and diagonals.

Take $row_1$ as example:

We can split this constraint *Exactly one queen per row* into two sub-constraints: "at most one" and "at least one" queen in each row.

*At most one queen per row*

$$p = \lceil \sqrt{m} \rceil = \lceil \sqrt{4} \rceil = 2$$

$$q = \left\lceil \frac{m}{p} \right\rceil = \left\lceil \frac{4}{2} \right\rceil = 2$$

So:

| | |
|---|---|
| $B_{00}$ | $B_{01}$ |
| $B_{02}$ | $B_{03}$ |

$(r_1 \rightarrow \neg r_2) \wedge$

$(c_1 \rightarrow \neg c_2) \wedge$

$(B_{00} \rightarrow r_0) \wedge (B_{00} \rightarrow c_0) \wedge$

$(B_{01} \rightarrow r_0) \wedge (B_{01} \rightarrow c_1) \wedge$

$(B_{02} \rightarrow r_1) \wedge (B_{02} \rightarrow c_0) \wedge$

$(B_{03} \rightarrow r_1) \wedge (B_{01} \rightarrow c_1) \wedge$

*At least one queen per row*

$B_{00} \vee B_{01} \vee B_{02} \vee B_{03}$

# Chapter 4

# Experiments & Result

In this chapter, the report compares and evaluates the performance of the *Binomial* method (which has been previously published) against the methods of optimizing the number of clauses: *Binary*, *Sequential counter*, *Product*, and *Commander* Encoding.

## 4.1 Datasets

In the N-Queens problem, the input consists of various board sizes (NxN) on which queens must be placed so that no two queens can attack each other. Specifically, the set of board sizes is determined by taking the squares and cubes of integers ranging from 2 to 11. This results in a diverse set of board dimensions that challenge the solver to find valid queen placements on boards of varying complexity.

*For example:*

n=4 then the solution is

$$\begin{bmatrix} * & Q & * & * \\ * & * & * & Q \\ Q & * & * & * \\ * & * & Q & * \end{bmatrix}$$

## 4.2 Experimental Environments

Table 4.1: Configuration of the experimental environment

| | |
|---|---|
| **Processor** | 11th Gen Intel(R) Core(TM) i7-1165G7 |
| **No. of Cores** | 4 |
| **RAM** | 16 GB |
| **SSD** | 512 GB |
| **SAT Solver** | Glucose3 |
| **Timeout** | 100 seconds |

Table 4.2: The runtime of different SAT encoding methods (seconds)

| **n** | **Binomial** | **Binary** | **Sequential counter** | **Commander** | **Product** |
|---|---|---|---|---|---|
| 4 | 0.0011 | 0.0020 | 0.0157 | 0.0013 | **0.0010** |
| 8 | **0.0010** | 0.0156 | 0.0156 | 0.0022 | 0.0025 |
| 9 | **0.0023** | 0.0156 | 0.0156 | 0.0044 | 0.0052 |
| 16 | **0.0141** | 0.0389 | 0.0287 | 0.0459 | 0.5128 |
| 25 | **0.0249** | 0.1993 | 0.0706 | 0.0343 | 17.6870 |
| 27 | **0.0370** | 0.5222 | 0.9674 | 0.0807 | 22.4735 |
| 36 | 0.0762 | 0.1499 | 0.2469 | **0.0609** | 0.2402 |
| 49 | 0.1826 | 0.1963 | 0.3260 | 0.2076 | **0.1175** |
| 64 | 0.4168 | 0.3068 | 44.3678 | 0.3290 | **0.2173** |
| 81 | 1.1081 | 0.3832 | 1.6676 | **0.3626** | 0.6378 |
| 100 | 2.2886 | 0.6154 | 91.5879 | 0.5575 | **0.3341** |
| 125 | 3.4561 | 1.1887 | *timeout* | 1.2111 | **0.8783** |
| 216 | 19.2357 | 2.9765 | *timeout* | 3.2802 | **1.4351** |
| 343 | 129.0981 | 7.8920 | *timeout* | 16.2076 | **3.8332** |
| 512 | *timeout* | 17.6089 | *timeout* | 19.3059 | **9.6076** |
| 729 | *timeout* | *timeout* | *timeout* | 79.9434 | **20.9605** |
| 1000 | *timeout* | *timeout* | *timeout* | 80.7082 | **47.9359** |
| 1331 | *timeout* | *timeout* | *timeout* | *timeout* | *timeout* |

Based on the *Table 4.2*, we can compare the performance of different SAT encoding methods for the N-Queens problem. For small values of $n$, such as 4 and 8, all methods perform efficiently with minimal runtime, suggesting they are well-suited for smaller problem sizes. The *Binomial* encoding method shows the lowest runtimes across these smaller values, with an increasing trend as $n$ grows. For instance, it has runtimes of 0.0011 seconds for $n = 4$ and 0.0010 seconds for $n = 8$. The *Binary* encoding also performs well initially but shows a noticeable increase in runtime as $n$ progresses, with 0.0020 seconds for $n = 4$ and 0.0156 seconds for $n = 8$. *Sequential counter* method has slightly higher runtimes for small $n$ values, with 0.0157 seconds

for $n = 4$ and 0.0156 seconds for $n = 8$. As $n$ increases, this method's runtime becomes significantly higher, especially for $n = 64$ where it reaches 44.3678 seconds. The *Commander* method starts with competitive runtimes but increases more substantially with larger $n$, reaching 0.5575 seconds for $n = 100$. The *Product* encoding method, on the other hand, initially shows low runtimes but dramatically increases with larger $n$, peaking at 47.9359 seconds for $n = 1000$. For larger values of $n$, all methods encounter significant performance issues, with *timeouts* becoming prevalent for $n \geq 125$ in most cases. The *Product* method, while having an increasing runtime, still performs better than the other methods for very large $n$, suggesting it might handle larger problem sizes better despite its growing execution time. In summary, for smaller $n$, *Binomial* encoding is the fastest, while for larger $n$, the *Product* encoding method exhibits better performance relative to the other methods despite its eventual *timeout*.

The number of logical variables and clauses are key factors influencing the performance of SAT Encoding methods. Therefore, in addition to the number of solvable matrices and solving time, the number of variables and clauses generated are also crucial aspects to consider for this method.

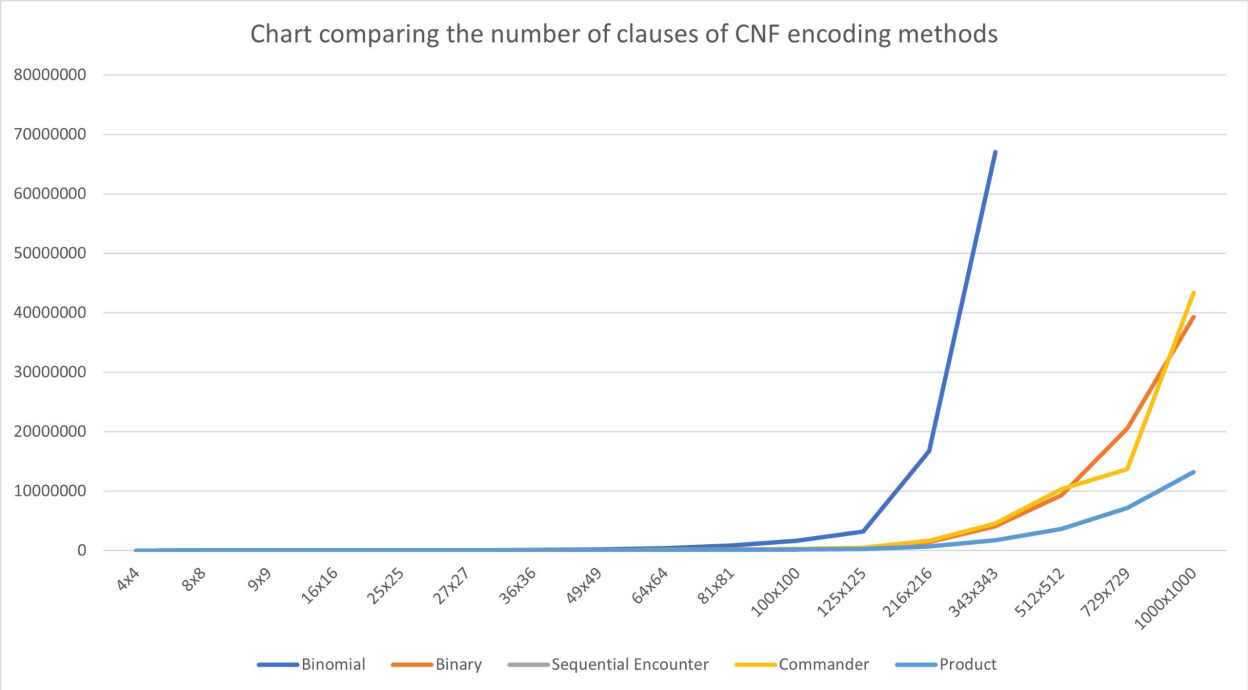



Figure 4.1: Comparing the number of clauses of different CNF encoding methods

For smaller board sizes, such as 4x4 and 8x8, the number of clauses is relatively low, with the *Binomial* and *Sequential counter* methods producing fewer clauses compared to *Binary* and *Commander* methods. As the board size increases, the number of clauses grows significantly. For instance, at 16x16, the number of clauses for the *Bi-*

*nomial* encoding is substantially higher than for the *Binary* encoding, and this trend continues as board sizes increase. The *Commander* and *Product* methods generally produce a higher number of clauses compared to the other methods, particularly for larger board sizes like 49x49 and 64x64. Notably, for very large board sizes such as 729x729 and 1000x1000, the *Binary* encoding method results in the highest number of clauses, indicating a rapid growth in complexity. This suggests that the *Binary* method may be less efficient for larger N-Queens problems compared to other encodings. In contrast, the *Commander* and *Product* methods, while producing a large number of clauses, show a relatively slower increase compared to *Binary*.

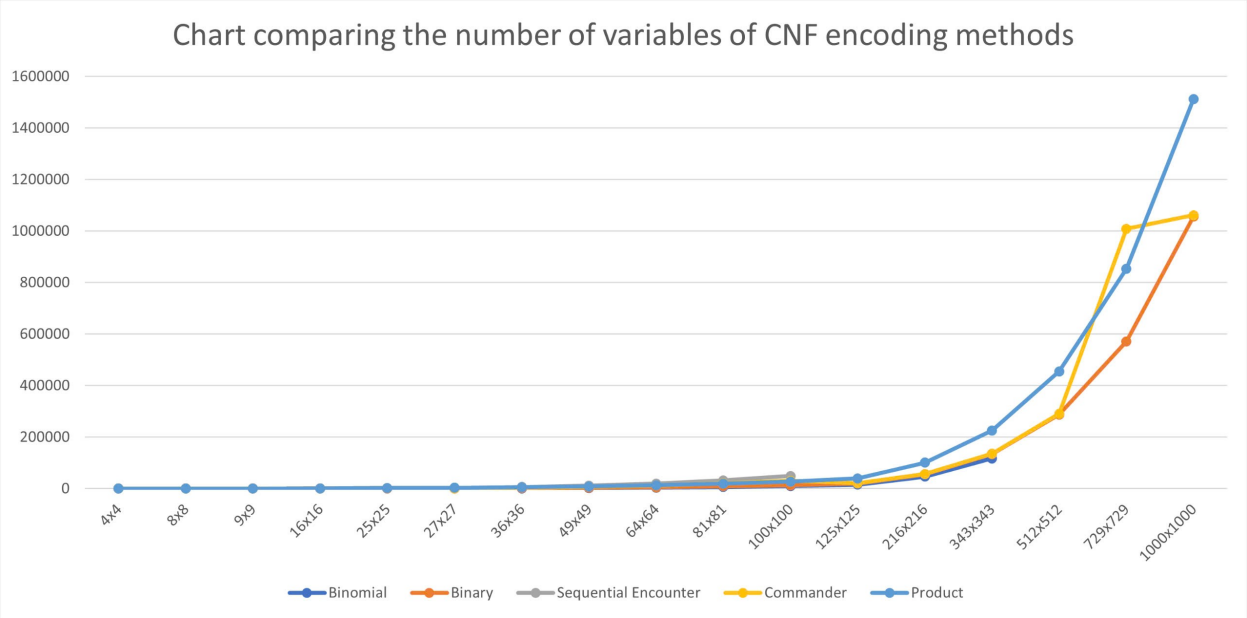



Figure 4.2: Comparing the number of variables of different CNF encoding methods

Initially, for smaller board sizes like 4x4 and 8x8, the number of variables is relatively low, with the *Binomial* method generating the fewest variables. However, as the board size increases, all methods show significant growth in the number of variables. The *Binary* and *Commander* methods consistently generate a higher number of variables compared to *Binomial*, especially noticeable in larger boards like 64x64 and above. The *Sequential counter* and *Product* methods also exhibit a rapid increase in variables, with the *Product* method showing particularly high numbers for very large boards, such as 343x343 and 1000x1000. This trend indicates that while more variables might suggest a more powerful or expressive encoding, it also implies greater complexity and potential computational overhead. As board sizes grow, the *Binary*, *Commander*, and *Product* methods demonstrate their ability to scale up in the number of variables, potentially offering more flexibility or detail in the encoding but at the cost of increased computational resources.

# Chapter 5

# Conclusion

In this report, I have proposed using optimized CNF encoding methods to enhance the performance of solving large-sized matrices in the N-Queens problem. The experimental results are quite promising, as these optimized encodings have shown superior performance compared to the previously published Binomial encoding.

In conclusion, the *Product* encoding method has proven to be the most effective in managing the trade-off between the number of variables and the number of clauses generated, particularly for large board sizes. It consistently maintains a balanced increase in variables while keeping the growth in the number of clauses under control, leading to faster solving times. This efficiency becomes especially evident in very large board sizes, where the *Product* encoding outperforms other encodings in both speed and scalability. The ability of the *Product* encoding to optimize these two critical factors makes it the superior choice for solving large-scale N-Queens problems. As a result, it stands out as the most efficient SAT encoding approach for handling complex, large-sized matrices. However, even this method fails to find a solution within the 100-second timeout when $n = 1331$.

Looking forward, future work could involve exploring other encoding methods to further enhance the efficiency of solving the N-Queens problem. Additionally, conducting experiments using the PBLib (Pseudo-Boolean Optimization) library [16] for solving At-Most-One (AMO) constraints could provide valuable insights and potentially improve performance even further.

# References


[1] Gilles Audemard and Laurent Simon, *On the glucose sat solver*, International Journal on Artificial Intelligence Tools **27** (2018), no. 01, 1840001.

[2] Federico Barber and Miguel A Salido, *Introduction to constraint programming*, Inteligencia Artificial, Revista Iberoamericana de Inteligencia Artificial **20** (2003), 13–30.

[3] Armin Biere, *Lingeling, plingeling, picosat and precosat at sat race 2010*, (2010).

[4] Roberto Campos, Pablo AM Casares, and MA Martin-Delgado, *Quantum metropolis solver: a quantum walks approach to optimization problems*, Quantum Machine Intelligence **5** (2023), no. 2, 28.

[5] Edmund M Clarke and Jeannette M Wing, *Formal methods: State of the art and future directions*, ACM Computing Surveys (CSUR) **28** (1996), no. 4, 626–643.

[6] GeeksforGeeks, *N queen problem | backtracking-3*, https://www.geeksforgeeks.org/n-queen-problem-backtracking-3/, Accessed: 2024-08-14.

[7] ———, *N queen problem | branch and bound*, https://www.geeksforgeeks.org/n-queen-problem-using-branch-and-bound/, Accessed: 2024-08-14.

[8] Karla L Hoffman, Manfred Padberg, Giovanni Rinaldi, et al., *Traveling salesman problem*, Encyclopedia of operations research and management science **1** (2013), 1573–1578.

[9] Jeremy Johnson, *Satisfiability and sat solvers*, CS 270 Math Foundations of CS, pp. 24–42.

[10] Vishal Kesri and Manoj Kumar Mishra, *A new approach to solve n-queens problem based on series*, International Journal of Engineering, Research and Applications **3** (2013), no. 3, 1349–1349.

[11] To Van Khanh, *Binary, sequential encounter, commander, product encoding*, At Most One – AMO, At Least One – ALO Encoding, pp. 5–17.

[12] Daniel Le Berre and Anne Parrain, *The sat4j library, release 2.2*, Journal on Satisfiability, Boolean Modeling and Computation **7** (2010), no. 2-3, 59–64.

[13] Craig Letavec and John Ruggiero, *The n-queens problem*, INFORMS Transactions on Education **2** (2002), no. 3, 101–103.

[14] Joao Marques-Silva, *Practical applications of boolean satisfiability*, 2008 9th International Workshop on Discrete Event Systems, IEEE, 2008, pp. 74–80.

[15] Matthew W Moskewicz, Conor F Madigan, Ying Zhao, Lintao Zhang, and Sharad Malik, *Chaff: Engineering an efficient sat solver*, Proceedings of the 38th annual Design Automation Conference, 2001, pp. 530–535.

[16] Tobias Philipp and Peter Steinke, *Pblib–a library for encoding pseudo-boolean constraints into cnf*, International Conference on Theory and Applications of Satisfiability Testing, Springer, 2015, pp. 9–16.

[17] Igor Rivin, Ilan Vardi, and Paul Zimmermann, *The n-queens problem*, The American Mathematical Monthly **101** (1994), no. 7, 629–639.

[18] Laurent Simon, *Reasoning with propositional logic: From sat solvers to knowledge compilation*, A Guided Tour of Artificial Intelligence Research: Volume II: AI Algorithms (2020), 115–152.

[19] Rok Sosic and Jun Gu, *Efficient local search with conflict minimization: A case study of the n-queens problem*, IEEE Transactions on Knowledge and Data Engineering **6** (1994), no. 5, 661–668.